

SonarQube calcule surtout :

Score	Ce qu'il mesure	Impact
Reliability (Fiabilité)	Bugs détectés	Les bugs critiques font beaucoup baisser le score.
Security (Sécurité)	Vulnérabilités et Security Hotspots	Les vulnérabilités critiques font beaucoup baisser le score.
Maintainability (Maintenabilité)	Code Smells	Chaque code smell diminue le score selon sa gravité (mineur, majeur, critique).

Tableau récapitulatif — Types d'erreurs / Métriques / Score

Métrique (Dashboard)	Type d'erreurs inclus	Ce que mesure la métrique	Seuil attendu (projet sain)
Reliability	Bugs	Risque de plantage, comportement inattendu, erreurs runtime	Note A (0 bug critique)
Security	Vulnérabilités	Niveau réel de sécurité du code contre attaques (XSS, injections SQL, accès non protégé...)	Note A (0 vulnérabilité)
Security Hotspots (revues manuelles)	Hotspots – zones sensibles nécessitant audit humain	Code sensible qui peut devenir vulnérable selon contexte (ex : crypto, droits fichiers, HTTP headers...)	Tous les hotspots doivent être "Reviewed"
Maintainability	Code Smells	État de propreté, lisibilité, dette technique	Note A (aucun smell majeur)
Coverage	Tests insuffisants	% du code exécuté par les tests unitaires automatisés	≥ 80 %
Duplications	Code dupliqué	Pourcentage de lignes dupliquées dans le projet	< 3 %
Quality Gate (verdict final)	Synthèse de toutes les métriques	Décision finale du statut qualité du projet	Status : PASSED

Méthode simple pour “calculer” un score à la main

1. Attribue un poids à chaque problème

Par exemple :

Gravité	Poids pour le score
Bloquant / Critique	5
Majeur	3
Mineur	1

2. Compter le nombre de problèmes par catégorie

- Bugs → Reliability
- Vulnerabilities → Security
- Code smells → Maintainability

3. Calculer un score brut

On peut faire simple :

$$Score = 100 - \frac{\text{Poids total des problèmes}}{\text{Poids max possible}} \times 100$$

Exemple pour Maintainability :

- 10 code smells mineurs → poids total = $10 \times 1 = 10$
- Si on considère que 50 points est le max de problèmes → $Score = 100 - (10/50 \times 100) = 80$

4. Arrondir ou transformer en lettre (A à E)

- 85-100 → A
- 70-84 → B
- 55-69 → C
- 40-54 → D
- <40 → E

Exercices pratiques :

Exercice 1 :

Voici un petit projet Redux Toolkit pour gérer un **todo list** :

```
// todosSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  todos: []
};

export const todosSlice = createSlice({
  name: 'todos',
  initialState,
  reducers: {
    addTodo: (state, action) => {
      state.todos.push({ id: state.todos.length + 1, text: action.payload, done: false });
    },
    toggleTodo: (state, action) => {
      const todo = state.todos.find(t => t.id === action.payload);
      todo.done = !todo.done;
    },
    removeTodo: (state, action) => {
```

```

    state.todos = state.todos.filter(t => t.id !== action.payload);
  },
  dangerousUpdate: (state, action) => {
    eval(action.payload); // Mauvaise pratique
  }
},
});

export const { addTodo, toggleTodo, removeTodo, dangerousUpdate } = todosSlice.actions;
export default todosSlice.reducer;

```

Questions :

1. Identifie tous les **types de problèmes** (Bug, Code Smell, Vulnerability).
2. Attribue une **gravité** (mineur, majeur, critique).
3. Calcule les **scores à la main** avec le tableau précédent.

Correction et calcul

Étape 1 : Identifier les problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	exécution dangereuse	Vulnerability	Critique (5)
todo.done = !todo.done sans vérifier si todo existe	Risque d'erreur si ID inexistant	Bug	Majeur (3)
state.todos.push(...) sans validation	Potentiel problème de type	Code Smell	Mineur (1)

Étape 2 : Calcul des scores (Poids max = 10 pour simplifier)

- **Security** = $100 - (5 / 10 \times 100) = 50 \rightarrow \mathbf{D}$
- **Reliability** = $100 - (3 / 10 \times 100) = 70 \rightarrow \mathbf{B}$
- **Maintainability** = $100 - (1 / 10 \times 100) = 90 \rightarrow \mathbf{A}$

Exercice 2 : Counter Slice

```

// counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { value: 0 };

export const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment: (state) => { state.value += 1; },
  },
});

```

```

decrement: (state) => { state.value -= 1; },
unsafeUpdate: (state, action) => {
  eval(action.payload); //Vulnérabilité critique
}
},
});

export const { increment, decrement, unsafeUpdate } = counterSlice.actions;
export default counterSlice.reducer;

```

Identification des problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Exécution dangereuse	Vulnerability	Critique (5)

Score (Poids max = 10)

- Security = $100 - (5/10 \times 100) = 50 \rightarrow \mathbf{D}$
- Reliability = 100 $\rightarrow \mathbf{A}$
- Maintainability = 100 $\rightarrow \mathbf{A}$

Exercice 3 : Todo Slice

```

// todosSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { todos: [] };

export const todosSlice = createSlice({
  name: 'todos',
  initialState,
  reducers: {
    addTodo: (state, action) => {
      state.todos.push({ id: state.todos.length + 1, text: action.payload }); // Code Smell mineur
    },
    toggleTodo: (state, action) => {
      const todo = state.todos.find(t => t.id === action.payload);
      todo.done = !todo.done; // Bug si todo est undefined
    }
  },
});

export const { addTodo, toggleTodo } = todosSlice.actions;
export default todosSlice.reducer;

```

Identification des problèmes

Ligne	Problème	Type	Gravité
todo.done = !todo.done	Risque d'erreur si ID inexistant	Bug	Majeur (3)
state.todos.push(...)	Pas de validation	Code Smell	Mineur (1)

Score (Poids max = 10)

- Reliability = $100 - (3/10 \times 100) = 70 \rightarrow \mathbf{B}$
- Maintainability = $100 - (1/10 \times 100) = 90 \rightarrow \mathbf{A}$

- Security = 100 → **A**

Exercice 4 : User Slice

```
// userSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { users: [] };

export const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    addUser: (state, action) => {
      state.users.push(action.payload); // Code Smell mineur
    },
    deleteUser: (state, action) => {
      state.users = state.users.filter(u => u.id !== action.payload); // Bug si action.payload undefined
    },
    updateUserUnsafe: (state, action) => {
      eval(action.payload); // Vulnérabilité critique
    }
  },
})
export const { addUser, deleteUser, updateUserUnsafe } = userSlice.actions;
export default userSlice.reducer;
```

Identification des problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Code dangereux	Vulnerability	Critique (5)
deleteUser	Risque d'erreur	Bug	Majeur (3)
state.users.push(action.payload)	Pas de validation	Code Smell	Mineur (1)

Score (Poids max = 10)

- Security = 100 - (5/10 × 100) = 50 → **D**
- Reliability = 100 - (3/10 × 100) = 70 → **B**
- Maintainability = 100 - (1/10 × 100) = 90 → **A**

Exercice 5 : Cart Slice

```
// cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { items: [] };

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addItem: (state, action) => { state.items.push(action.payload); }, // Code Smell mineur
    removeItem: (state, action) => { state.items = state.items.filter(i => i.id !== action.payload); }, //
```

Bug possible

```
unsafeClear: (state, action) => { eval(action.payload); } // Vulnérabilité critique
},
});
```

```
export const { addItem, removeItem, unsafeClear } = cartSlice.actions;
export default cartSlice.reducer;
```

Identification des problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Danger	Vulnerability	Critique (5)
removeItem	Risque d'erreur si payload invalide	Bug	Majeur (3)
addItem	Pas de validation	Code Smell	Mineur (1)

Score (Poids max = 10)

- Security = 50 → **D**
- Reliability = 70 → **B**
- Maintainability = 90 → **A**

Exercice 6 : Settings Slice

```
// settingsSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { theme: 'light' };

export const settingsSlice = createSlice({
  name: 'settings',
  initialState,
  reducers: {
    toggleTheme: (state) => { state.theme = state.theme === 'light' ? 'dark' : 'light'; }, //
    Code Smell mineur
    resetThemeUnsafe: (state, action) => { eval(action.payload); } // Vulnérabilité critique
  },
});

export const { toggleTheme, resetThemeUnsafe } = settingsSlice.actions;
export default settingsSlice.reducer;
```

Identification des problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Danger	Vulnerability	Critique (5)
toggleTheme	Pas de contrôle sur type	Code Smell	Mineur (1)

Score (Poids max = 10)

- Security = 50 → **D**
- Reliability = 100 → **A**
- Maintainability = 90 → **A**

Exercice 7 : Auth Slice

```
// authSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { user: null, token: null };

export const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    login: (state, action) => { state.user = action.payload.user; state.token = action.payload.token; },
    logout: (state) => { state.user = null; state.token = null; },
    unsafeLogin: (state, action) => { eval(action.payload); } //Vulnérabilité critique
  },
});

export const { login, logout, unsafeLogin } = authSlice.actions;
export default authSlice.reducer;
```

Identification des problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Exécution dangereuse	Vulnerability	Critique (5)

Calcul des scores (Poids max = 10)

- Security = $100 - (5/10 \times 100) = 50 \rightarrow \mathbf{D}$
- Reliability = 100 $\rightarrow \mathbf{A}$
- Maintainability = 100 $\rightarrow \mathbf{A}$

Exercice 8 : Profile Slice

```
// profileSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { profiles: [] };

export const profileSlice = createSlice({
  name: 'profile',
  initialState,
  reducers: {
    addProfile: (state, action) => { state.profiles.push(action.payload); }, // Code Smell mineur
    updateProfile: (state, action) => {
      const profile = state.profiles.find(p => p.id === action.payload.id);
      profile.name = action.payload.name; // Bug si profile undefined
    }
  },
});

export const { addProfile, updateProfile } = profileSlice.actions;
export default profileSlice.reducer;
```

Identification des problèmes

Ligne	Problème	Type	Gravité
profile.name = action.payload.name	Risque d'erreur si ID inexistant	Bug	Majeur (3)
state.profiles.push(action.payload)	Pas de validation	Code Smell	Mineur (1)

Calcul des scores

- Reliability = $100 - (3/10 \times 100) = 70 \rightarrow \mathbf{B}$
- Maintainability = $100 - (1/10 \times 100) = 90 \rightarrow \mathbf{A}$
- Security = 100 $\rightarrow \mathbf{A}$

Exercice 9 : Settings Slice (Avancé)

```
// settingsSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { options: { darkMode: false } };

export const settingsSlice = createSlice({
  name: 'settings',
  initialState,
  reducers: {
    toggleDarkMode: (state) => { state.options.darkMode = !state.options.darkMode; }, // Code Smell mineur
    unsafeSettingsUpdate: (state, action) => { eval(action.payload); } // Vulnérabilité critique
  },
});

export const { toggleDarkMode, unsafeSettingsUpdate } = settingsSlice.actions;
export default settingsSlice.reducer;
```

Problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Danger	Vulnerability	Critique (5)
toggleDarkMode	Pas de validation	Code Smell	Mineur (1)

Scores

- Security = 50 $\rightarrow \mathbf{D}$
- Maintainability = 90 $\rightarrow \mathbf{A}$
- Reliability = 100 $\rightarrow \mathbf{A}$

Exercice 10 : Products Slice

```
// productsSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { products: [] };

export const productsSlice = createSlice({
  name: 'products',
  initialState,
  reducers: {
```



```

addProduct: (state, action) => { state.products.push(action.payload); }, // Code Smell mineur
deleteProduct: (state, action) => {
  const index = state.products.findIndex(p => p.id === action.payload);
  state.products.splice(index, 1); // Bug si index = -1
},
unsafeUpdate: (state, action) => { eval(action.payload); } // Vulnérabilité critique
},
});

export const { addProduct, deleteProduct, unsafeUpdate } = productsSlice.actions;
export default productsSlice.reducer;

```

Problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Danger	Vulnerability	Critique (5)
splice(index,1)	Bug si index invalide	Bug	Majeur (3)
push(action.payload)	Pas de validation	Code Smell	Mineur (1)

Scores

- Security = 50 → **D**
- Reliability = 70 → **B**
- Maintainability = 90 → **A**

Exercice 11 : Cart Slice (Avancé)

```

// cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = { items: [] };

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addItem: (state, action) => { state.items.push(action.payload); }, // Code Smell mineur
    removeItem: (state, action) => {
      const item = state.items.find(i => i.id === action.payload);
      item.count -= 1; // Bug si item undefined
    },
    unsafeClearCart: (state, action) => { eval(action.payload); } // Vulnérabilité critique
  },
});

export const { addItem, removeItem, unsafeClearCart } = cartSlice.actions;
export default cartSlice.reducer;

```

Problèmes

Ligne	Problème	Type	Gravité
eval(action.payload)	Danger	Vulnerability	Critique (5)

item.count -= 1	Bug si item inexistant	Bug	Majeur (3)
push(action.payload)	Pas de validation	Code Smell	Mineur (1)

Scores

- Security = 50 → **D**
- Reliability = 70 → **B**
- Maintainability = 90 → **A**

Vulnérabilités Vs Security Hotspots

Vulnérabilités (Vulnerabilities)

Définition simple :

Une vulnérabilité est un vrai problème de sécurité dans ton code, quelque chose qui peut être exploité immédiatement par un attaquant pour causer des dégâts.

Métaphore :

Imagine ta maison :

- Une porte de coffre complètement ouverte.
- N'importe qui peut entrer et voler tes objets de valeur.

Exemple concret en React/Redux Toolkit :

```
// Mauvaise pratique = vulnérabilité
eval(action.payload); // Danger : le code peut exécuter n'importe quoi
C'est un vrai danger → il faut corriger immédiatement.
```

Security Hotspots

Définition simple :

Un Security Hotspot est une zone du code qui pourrait devenir dangereuse si elle est mal utilisée, mais elle n'est pas forcément une vulnérabilité directe.

Il faut vérifier la logique et décider si c'est sûr.

Métaphore :

Imagine ta maison :

- Une fenêtre fragile ou une porte qui ferme mais sans verrou solide.
- Ce n'est pas exploitable tout de suite, mais il faut la surveiller et décider si tu dois ajouter un verrou.

Exemple concret en React/Redux Toolkit :

```
// Security Hotspot
const token = localStorage.getItem('authToken');
```

```
fetch('/api/data', { headers: { Authorization: token } });
```

- Ce n'est pas un bug direct.
- Mais si le token est stocké de manière non sécurisée, ça peut devenir une vulnérabilité → à vérifier.

Différence clé résumée

Concept	Danger immédiat ?	Action requise	Exemple
Vulnerability	Oui, exploitable maintenant	Corriger tout de suite	eval(action.payload)
Security Hotspot	Pas forcément	Vérifier et décider si sécuriser	Utilisation d'un token stocké en localStorage

Astuce pour retenir :

- **Vulnérabilité = feu déjà allumé** → il faut éteindre maintenant
- **Security Hotspot = allumette posée près du feu** → à surveiller avant que ça devienne un feu